

# 11 — Graphs

DSA in Python — Interview Prep

---

## Graphs — Tutorial

Covers the 10 problems solved in this folder. A graph problem is almost always one of two underlying questions dressed up differently: "what can I reach from here" (traversal/connectivity) or "in what order must these things happen" (ordering under dependency constraints). Grids are graphs in disguise — each cell is a node, each 4-directional neighbor relationship is an edge — which is why this folder starts on grids before moving to explicit adjacency-list graphs.

### The patterns

#### 1. Grid DFS — the base template (01, 02, 03)

One guard clause (bounds check + validity check) at the top of the recursive call, then recurse to all 4 neighbors. What differs across these three problems is only \*what the DFS accomplishes\*:

- `flood_fill`: mutates cells as it visits them.
- `number_of_islands`: uses a separate `visited` set (keeps input unmodified) and counts how many times a \*new\* DFS had to start from an unvisited cell — that count is the number of components.
- `max_area_of_island`: the DFS \*returns a value\* (1 + sum of what each neighbor call returns) instead of just marking cells — the same "combine children's answers" recursive shape from the trees topic, applied to a grid's implicit graph.

#### 2. Multi-source BFS (04)

`rotting_oranges` needs \*minimum time\*, which requires BFS, not DFS — BFS naturally processes cells in order of distance from a source, which corresponds exactly to elapsed time here. "Multi-source" means seeding the queue with \*every\* initially-rotten orange at once (distance/time 0 for all of them simultaneously), not just one starting point — the queue then expands the rot outward from all sources in lockstep, and the last time popped is the total time needed.

#### 3. Traversing an explicit graph with cycles (05)

`clone_graph` is the first problem here on an explicit adjacency-list graph (not a grid), and the first where a \*plain\* DFS would infinite-loop without a visited check, since undirected neighbor relationships create cycles by nature. The `original -> clone` dict does double duty: it's the visited-check (already a key means already fully handled) and it's how neighbor references get wired to the correct existing clone instead of creating duplicates.

#### 4. Component counting on an explicit graph (06)

Directly generalizes 02's island-counting from a grid's implicit adjacency to an explicit edge list: build an adjacency list from the edges once, then the exact same "scan for unvisited nodes, DFS from each one found, count how many times a new DFS started" logic applies unchanged.

#### 5. Directed-graph cycle detection with 3 states (07)

Undirected cycle detection (09) only needs visited/unvisited, because re-encountering *any* visited neighbor (other than the immediate parent) means a cycle. Directed graphs need a third state: unvisited, *currently on the recursion stack* ("visiting"), and fully done. A cycle exists specifically when DFS reaches a node still in the "visiting" state — a back edge to an ancestor on the current path — not just any previously-visited node, since two different paths in a DAG can legitimately converge on an already-*done* node without that being a cycle.

## 6. Topological sort via BFS on in-degree (08)

Kahn's algorithm answers "in what order" using the same "peel off what's currently unblocked" idea as 07's cycle check, but built on in-degree counting and BFS instead of DFS coloring. Nodes with in-degree 0 have no unmet dependencies, so they're immediately takeable; taking one conceptually removes it from the graph by decrementing its neighbors' in-degrees, and any neighbor that drops to 0 becomes takeable in turn. If the final processed count is less than the total node count, some nodes were stuck depending on each other in a cycle and never reached in-degree 0 — this is actually an alternative way to detect the same cycles 07 finds via coloring.

## 7. Undirected connectivity + acyclicity together (09)

`graph_valid_tree` needs both "no cycle" and "fully connected" to hold simultaneously. The `edges == n - 1` check is a free  $O(1)$  filter (a tree-shaped graph has exactly that many edges by definition); the DFS then only needs to check for cycles (skip the immediate parent — that's just the edge you arrived on, not a real cycle — but treat any *other* already-visited neighbor as one) and confirm every node got reached.

## 8. Two multi-source traversals instead of one per cell (10)

`pacific_atlantic_water_flow`'s brute-force framing ("does water from this cell reach both oceans") would need one flood-fill per cell. Reversing the question — "which cells could water flow backward into from the ocean" — turns it into exactly two multi-source traversals (one seeded from all Pacific-border cells, one from all Atlantic-border cells), with the flow condition also reversed (walking toward *higher or equal* ground instead of lower, since that's what "backward" means here). The answer is just the intersection of the two visited sets. This "run the traversal backward from the target instead of forward from every possible source" reframing is a generally useful trick whenever a problem asks about reachability *to* something from *many* possible starting points.

## Complexity summary

| #  | Problem                        | Time                               | Extra space                                        |
|----|--------------------------------|------------------------------------|----------------------------------------------------|
| 01 | flood_fill                     | $O(\text{rows} \cdot \text{cols})$ | $O(\text{rows} \cdot \text{cols})$ recursion depth |
| 02 | number_of_islands              | $O(\text{rows} \cdot \text{cols})$ | $O(\text{rows} \cdot \text{cols})$                 |
| 03 | max_area_of_island             | $O(\text{rows} \cdot \text{cols})$ | $O(\text{rows} \cdot \text{cols})$                 |
| 04 | rotting_oranges                | $O(\text{rows} \cdot \text{cols})$ | $O(\text{rows} \cdot \text{cols})$                 |
| 05 | clone_graph                    | $O(V+E)$                           | $O(V)$                                             |
| 06 | number_of_connected_components | $O(V+E)$                           | $O(V+E)$                                           |
| 07 | course_schedule                | $O(V+E)$                           | $O(V+E)$                                           |
| 08 | course_schedule_ii             | $O(V+E)$                           | $O(V+E)$                                           |
| 09 | graph_valid_tree               | $O(V+E)$                           | $O(V+E)$                                           |

| #  | Problem                                  | Time                               | Extra space                        |
|----|------------------------------------------|------------------------------------|------------------------------------|
| 10 | <code>pacific_atlantic_water_flow</code> | $O(\text{rows} \cdot \text{cols})$ | $O(\text{rows} \cdot \text{cols})$ |

Every problem here is linear in graph/grid size ( $V+E$ , or  $\text{rows} \cdot \text{cols}$ ) — the recurring interview expectation for BFS/DFS-based graph problems. If a proposed solution is worse than that (e.g. re-traversing from every cell independently, as the brute-force framing of `10` would), that's usually the signal to look for a multi-source or reversed-direction reframing instead.

## Common interview follow-ups to be ready for

- `number_of_islands` / `max_area_of_island`: "can you do it with BFS instead of DFS?" — yes, swap the recursive calls for a queue; the visited-tracking logic is identical either way.
- `rotting_oranges`: "why not DFS?" — be ready to state precisely: DFS would reach some fresh oranges via a long path before reaching closer ones via a short path, giving wrong (too-large) times; BFS's level-by-level property is what guarantees correct minimums.
- `clone_graph`: "what if the graph were directed instead of undirected?" — the same dict-based DFS still works unchanged, since the visited-check doesn't depend on edges being bidirectional.
- `course_schedule` / `course_schedule_ii`: "DFS coloring vs. Kahn's BFS - which do you prefer and why?" — both are  $O(V+E)$ ; Kahn's is often preferred when the actual order is needed (it falls out naturally), while DFS coloring is often more direct when only a yes/no cycle answer is needed.
- `graph_valid_tree`: "what if the graph could be disconnected with extra edges, i.e. not exactly  $n-1$  edges?" — the  $O(1)$  edge-count filter no longer applies; would need to check connectivity AND acyclicity as two fully separate conditions instead of getting one for free from the other.
- `pacific_atlantic_water_flow`: "what if there were three oceans (or  $k$  of them)?" — the two-traversal-then-intersect approach generalizes cleanly to  $k$  traversals intersected together.

## Self-check before moving on

You should be able to, without looking at the code:

- Write the grid-DFS guard clause (bounds + validity check) from memory and explain why it belongs in one place, not repeated per direction.
- Explain why `rotting_oranges` must use BFS and what breaks with DFS.
- State the 3-state coloring used for directed-cycle detection and explain why "already fully done" is different from "currently on the path" when deciding if a cycle was found.
- Explain Kahn's algorithm's in-degree-decrement step and why a node becomes takeable exactly when its in-degree hits 0.
- Explain the "traverse backward from the target" reframing used in `pacific_atlantic_water_flow` and why it avoids one traversal per cell.

If any of those aren't fluent yet, this topic isn't "done" per the README's definition — revisit before moving to `12_graphs_advanced`.

# Programs

## 01. Flood Fill

01\_flood\_fill.py

### *Explanation*

Problem: Given an image (2D grid of colors) and a starting pixel (sr, sc), change the color of that pixel and every pixel connected to it (4-directionally) that shares its *original* color, to a new color.

Source : LeetCode 733 - Flood Fill

Example:

Input: image = [[1,1,1],[1,1,0],[1,0,1]], sr=1, sc=1, color=2

Output: [[2,2,2],[2,2,0],[2,0,1]]

Idea: the simplest possible grid DFS, and the template every other grid problem in this folder builds on. Recurse to the 4 neighbors, but only continue into a cell if it's in bounds *and* still has the original color - both conditions belong in one guard clause at the top of the recursive call, so every call site (all 4 directions) gets the same bounds/color check for free rather than repeating it before each recursive call. The check for `original == color` up front prevents infinite recursion in the edge case where the fill color equals the starting color already.

### *Code*

```
def flood_fill(image: list[list[int]], sr: int, sc: int, color: int) -> list[list[int]]:
    original = image[sr][sc]
    if original == color:
        return image

    rows, cols = len(image), len(image[0])

    def dfs(r: int, c: int) -> None:
        if r < 0 or r >= rows or c < 0 or c >= cols or image[r][c] != original:
            return
        image[r][c] = color
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    dfs(sr, sc)
    return image

if __name__ == "__main__":
    tests = [
        ([[1, 1, 1], [1, 1, 0], [1, 0, 1]], 1, 1, 2,
         [[2, 2, 2], [2, 2, 0], [2, 0, 1]]),
        ([[0, 0, 0], [0, 0, 0]], 0, 0, 0,
```

```

        [[0, 0, 0], [0, 0, 0]]),
        ([[1]], 0, 0, 3, [[3]]),
    ]

    for i, (image, sr, sc, color, expected) in enumerate(tests, 1):
        got = flood_fill([row[:] for row in image], sr, sc, color)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

## 02. Number Of Islands

02\_number\_of\_islands.py

### Explanation

Problem: Given a grid of '1' (land) and '0' (water), return the number of islands (maximal 4-directionally connected groups of land).

Source : LeetCode 200 - Number of Islands

Example:

Input:

11110

11010

11000

00000

Output: 1

Idea: this is 01's flood fill technique used to \*count connected components\* rather than to recolor them. Scan every cell; whenever land is found that hasn't been visited yet, it's the start of a new island - increment the count and DFS outward marking every reachable land cell as visited, so the same island is never counted again from one of its other cells later in the scan. A **visited** set (rather than mutating the grid itself, as 01 does) keeps the input unmodified, which matters here since nothing about "counting" implies permission to change the input.

### Code

```

def num_islands(grid: list[list[str]]) -> int:
    rows, cols = len(grid), len(grid[0])
    visited = set()

    def dfs(r: int, c: int) -> None:
        if r < 0 or r >= rows or c < 0 or c >= cols:
            return
        if grid[r][c] == "0" or (r, c) in visited:
            return
        visited.add((r, c))
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    count = 0
    for r in range(rows):
        for c in range(cols):

```

```

        if grid[r][c] == "1" and (r, c) not in visited:
            count += 1
            dfs(r, c)

    return count

if __name__ == "__main__":
    tests = [
        ([
            list("11110"),
            list("11010"),
            list("11000"),
            list("00000"),
        ], 1),
        ([
            list("11000"),
            list("11000"),
            list("00100"),
            list("00011"),
        ], 3),
        ([list("0")], 0),
    ]

    for i, (grid, expected) in enumerate(tests, 1):
        got = num_islands(grid)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

## 03. Max Area Of Island

03\_max\_area\_of\_island.py

### *Explanation*

Problem: Given a grid of 1 (land) and 0 (water), return the area of the largest island (0 if there is none).

Source : LeetCode 695 - Max Area of Island

Example:

Input:

[[0,0,1,0,0],

[0,1,1,1,0],

[0,0,1,0,0]]

Output: 5

Idea: identical scan-and-DFS structure to 02, but the DFS now \*returns a value\* (the island's size) instead of just marking cells.

Each call contributes 1 for itself plus the sum of what all 4 neighbors contribute - the same "combine children's answers" recursive shape from the trees topic, just applied to a grid's implicit graph instead of an explicit tree. Track the best size seen across all islands found during the scan.

### *Code*

```

def max_area_of_island(grid: list[list[int]]) -> int:
    rows, cols = len(grid), len(grid[0])

```

```

visited = set()

def dfs(r: int, c: int) -> int:
    if r < 0 or r >= rows or c < 0 or c >= cols:
        return 0
    if grid[r][c] == 0 or (r, c) in visited:
        return 0
    visited.add((r, c))
    return 1 + dfs(r + 1, c) + dfs(r - 1, c) + dfs(r, c + 1) + dfs(r, c - 1)

best = 0
for r in range(rows):
    for c in range(cols):
        if grid[r][c] == 1 and (r, c) not in visited:
            best = max(best, dfs(r, c))

return best

if __name__ == "__main__":
    tests = [
        ([[0, 0, 1, 0, 0], [0, 1, 1, 1, 0], [0, 0, 1, 0, 0]], 5),
        ([[0, 0, 0], [0, 0, 0]], 0),
        ([[1, 1], [1, 1]], 4),
    ]

    for i, (grid, expected) in enumerate(tests, 1):
        got = max_area_of_island(grid)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

## 04. Rotting Oranges

04\_rotting\_oranges.py

### *Explanation*

Problem: A grid contains 0 (empty), 1 (fresh orange), or 2 (rotten orange). Every minute, every rotten orange rots its 4-directional fresh neighbors. Return the minimum minutes until no fresh orange remains, or -1 if that's impossible.

Source : LeetCode 994 - Rotting Oranges

Example:

Input:

[[2,1,1],

[1,1,0],

[0,1,1]]

Output: 4

Idea: multi-source BFS - the first pattern in this folder needing a queue instead of DFS. All initially-rotten oranges start in the queue *\*simultaneously\** (multiple sources, not one), each tagged with its rot-time (0 for the initial ones). BFS naturally processes cells in order of distance-from-a-source, which here directly corresponds to time-elapsed, so the last cell popped off the queue gives the total time needed. This is why BFS (not DFS) is required whenever a

problem asks for shortest-path/minimum-time on an unweighted grid or graph - DFS would explore in the wrong order to track "time" cleanly.

## Code

```
from collections import deque

def oranges_rotting(grid: list[list[int]]) -> int:
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c, 0))
            elif grid[r][c] == 1:
                fresh += 1

    minutes = 0
    while queue:
        r, c, t = queue.popleft()
        minutes = max(minutes, t)
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                grid[nr][nc] = 2
                fresh -= 1
                queue.append((nr, nc, t + 1))

    return minutes if fresh == 0 else -1

if __name__ == "__main__":
    tests = [
        ([[2, 1, 1], [1, 1, 0], [0, 1, 1]], 4),
        ([[2, 1, 1], [0, 1, 1], [1, 0, 1]], -1),
        ([[0, 2]], 0),
        ([[0, 0, 0]], 0),
    ]

    for i, (grid, expected) in enumerate(tests, 1):
        got = oranges_rotting([row[:] for row in grid])
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

## 05. Clone Graph

05\_clone\_graph.py

### Explanation

Problem: Given a reference to a node in a connected undirected graph (each node has a value and a list of neighbor references), return a deep copy of the entire graph.

Source : LeetCode 133 - Clone Graph

Example:

Input: a graph 1 -- 2

||

4 -- 3

Output: a structurally identical graph, entirely new node objects

Idea: the first problem in this folder on an explicit graph (nodes with neighbor lists) instead of an implicit grid. Cloning requires visiting every node exactly once despite cycles being possible (an undirected graph's neighbor relationships go both ways, so a plain DFS without tracking visited nodes would recurse forever). A dict mapping original node -> its clone serves two purposes at once: it's the visited-set (if a node is already a key, its clone already exists and traversal shouldn't continue past it) and it's how a node's neighbors get wired to the *\*already-created\** clones of whichever neighbors were visited first, rather than creating duplicate clones for the same original node.

## Code

```
class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []

def clone_graph(node: Node | None) -> Node | None:
    if not node:
        return None

    clones: dict[Node, Node] = {}

    def dfs(n: Node) -> Node:
        if n in clones:
            return clones[n]
        copy = Node(n.val)
        clones[n] = copy
        for neighbor in n.neighbors:
            copy.neighbors.append(dfs(neighbor))
        return copy

    return dfs(node)

def build_graph(adj_list: list[list[int]]) -> Node | None:
    if not adj_list:
        return None
    nodes = {i + 1: Node(i + 1) for i in range(len(adj_list))}
    for i, neighbor_vals in enumerate(adj_list):
        nodes[i + 1].neighbors = [nodes[v] for v in neighbor_vals]
    return nodes[1]

def to_adj_list(node: Node | None) -> list[list[int]]:
    if not node:
        return []
    visited: dict[int, Node] = {}

    def dfs(n: Node) -> None:
```

```

        if n.val in visited:
            return
        visited[n.val] = n
        for neighbor in n.neighbors:
            dfs(neighbor)

    dfs(node)
    ordered = [visited[v] for v in sorted(visited)]
    return [[nb.val for nb in n.neighbors] for n in ordered]

if __name__ == "__main__":
    tests = [
        [[2, 4], [1, 3], [2, 4], [1, 3]],
        [[]],
        [],
    ]

    for i, adj_list in enumerate(tests, 1):
        original = build_graph(adj_list)
        clone = clone_graph(original)
        got = to_adj_list(clone)
        status = "PASS" if got == adj_list else "FAIL"
        identity_ok = clone is not original if original else True
        status = status if identity_ok else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {adj_list})")

```

## 06. Number Of Connected Components

06\_number\_of\_connected\_components.py

### ***Explanation***

Problem: Given  $n$  nodes labeled 0 to  $n-1$  and a list of undirected edges, return the number of connected components.

Source : LeetCode 323 - Number of Connected Components in an Undirected Graph

Example:

Input:  $n = 5$ , edges =  $[[0, 1], [1, 2], [3, 4]]$

Output: 2

Idea: 02's island-counting logic, generalized from an implicit grid graph (neighbors = adjacent cells) to an explicit graph given as an edge list (neighbors = adjacency list built from those edges). Build the adjacency list once, then scan every node: whenever an unvisited node is found, it starts a new component - increment the count and DFS to mark every node reachable from it. Same "scan + DFS from unvisited starting points, count how many times a new DFS had to start" shape as 02, just on a general graph instead of a grid.

### ***Code***

```

def count_components(n: int, edges: list[list[int]]) -> int:
    adj: dict[int, list[int]] = {i: [] for i in range(n)}
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)

```

```

visited = set()

def dfs(node: int) -> None:
    for neighbor in adj[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            dfs(neighbor)

count = 0
for i in range(n):
    if i not in visited:
        visited.add(i)
        dfs(i)
        count += 1

return count

if __name__ == "__main__":
    tests = [
        (5, [[0, 1], [1, 2], [3, 4]], 2),
        (5, [[0, 1], [1, 2], [2, 3], [3, 4]], 1),
        (4, [], 4),
        (1, [], 1),
    ]

    for i, (n, edges, expected) in enumerate(tests, 1):
        got = count_components(n, edges)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

## 07. Course Schedule

07\_course\_schedule.py

### ***Explanation***

Problem: There are num\_courses courses (0 to num\_courses-1) and a list of prerequisite pairs [course, prereq] meaning prereq must be taken before course. Return True if it's possible to finish all courses (i.e. the prerequisite graph has no cycle).

Source : LeetCode 207 - Course Schedule

Example:

Input: num\_courses = 2, prerequisites = [[1, 0]]

Output: True (take 0, then 1)

Input: num\_courses = 2, prerequisites = [[1, 0], [0, 1]]

Output: False (0 needs 1, 1 needs 0 - impossible)

Idea: "can all courses be finished" is exactly "does this directed graph have a cycle" - if course A depends (transitively) on itself, no valid order can ever satisfy every prerequisite. Cycle detection on a *directed* graph needs 3 states per node, not the simple visited/unvisited boolean that works for undirected graphs (as in 06): unvisited (0), currently-being-explored / "on the current DFS path" (1), and fully-done (2). A cycle exists exactly when the DFS

reaches a node that's already in state 1 - a "back edge" to an ancestor on the current path. Reaching a node in state 2 is fine (it's a legitimately different path re-converging, not a cycle) and should short-circuit as "no cycle found through here" without re-exploring it.

## Code

```
def can_finish(num_courses: int, prerequisites: list[list[int]]) -> bool:
    adj: dict[int, list[int]] = {i: [] for i in range(num_courses)}
    for course, prereq in prerequisites:
        adj[course].append(prereq)

    UNVISITED, VISITING, DONE = 0, 1, 2
    state = [UNVISITED] * num_courses

    def has_cycle(node: int) -> bool:
        if state[node] == VISITING:
            return True
        if state[node] == DONE:
            return False

        state[node] = VISITING
        for neighbor in adj[node]:
            if has_cycle(neighbor):
                return True
        state[node] = DONE
        return False

    for course in range(num_courses):
        if has_cycle(course):
            return False

    return True

if __name__ == "__main__":
    tests = [
        (2, [[1, 0]], True),
        (2, [[1, 0], [0, 1]], False),
        (4, [[1, 0], [2, 0], [3, 1], [3, 2]], True),
        (3, [[0, 1], [1, 2], [2, 0]], False),
    ]

    for i, (num_courses, prereqs, expected) in enumerate(tests, 1):
        got = can_finish(num_courses, prereqs)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

## 08. Course Schedule II

08\_course\_schedule\_ii.py

### Explanation

Problem: Same setup as 07, but return a valid order to take all courses in, or [] if no valid order exists (a cycle).

Source : LeetCode 210 - Course Schedule II

Example:

Input: num\_courses = 4,

prerequisites = [[1,0],[2,0],[3,1],[3,2]]

Output: [0, 1, 2, 3] (or [0, 2, 1, 3] - either is valid)

Idea: Kahn's algorithm - topological sort via BFS instead of DFS, built on in-degree (how many prerequisites a course still has left unmet) rather than the 3-state coloring from 07. Any course with in-degree 0 has no unmet prerequisites, so it's safe to take right now - seed the queue with all such courses. Taking a course conceptually removes it from the graph, which is simulated by decrementing the in-degree of everything it was a prerequisite for; any neighbor whose in-degree drops to 0 as a result becomes takeable and joins the queue. If every course gets processed this way, the order they were dequeued in is a valid topological order; if some courses are left with in-degree > 0 at the end, they're stuck in a cycle with each other and no valid order exists.

## Code

```
from collections import deque

def find_order(num_courses: int, prerequisites: list[list[int]]) -> list[int]:
    adj: dict[int, list[int]] = {i: [] for i in range(num_courses)}
    indegree = [0] * num_courses

    for course, prereq in prerequisites:
        adj[prereq].append(course)
        indegree[course] += 1

    queue = deque(i for i in range(num_courses) if indegree[i] == 0)
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in adj[node]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                queue.append(neighbor)

    return order if len(order) == num_courses else []

def _is_valid_order(order, num_courses, prerequisites):
    if len(order) != num_courses:
        return False
    position = {course: i for i, course in enumerate(order)}
    return all(position[prereq] < position[course] for course, prereq in prerequisites)

if __name__ == "__main__":
    tests = [
        (4, [[1, 0], [2, 0], [3, 1], [3, 2]], True),
        (2, [[1, 0], [0, 1]], False),
        (1, [], True),
    ]
```

```

for i, (num_courses, prereqs, should_have_valid_order) in enumerate(tests, 1):
    got = find_order(num_courses, prereqs)
    if should_have_valid_order:
        status = "PASS" if _is_valid_order(got, num_courses, prereqs) else "FAIL"
    else:
        status = "PASS" if got == [] else "FAIL"
    print(f"Test {i}: {status} (got {got})")

```

## 09. Graph Valid Tree

09\_graph\_valid\_tree.py

### Explanation

Problem: Given  $n$  nodes labeled 0 to  $n-1$  and a list of undirected edges, determine if these edges form a valid tree (connected, and no cycles).

Source : LeetCode 261 - Graph Valid Tree

Example:

Input:  $n = 5$ , edges =  $[[0,1],[0,2],[0,3],[1,4]]$

Output: True

Input:  $n = 5$ , edges =  $[[0,1],[1,2],[2,3],[1,3],[1,4]]$

Output: False (1-2-3-1 is a cycle)

Idea: a tree with  $n$  nodes has exactly  $n-1$  edges, and any connected graph with exactly  $n-1$  edges is automatically acyclic (and vice versa) - so checking the edge count up front is a cheap  $O(1)$  filter that rules out most invalid cases before any traversal. What's left to verify is connectivity, via DFS from node 0: track the **parent** each node was reached from, and treat re-encountering an \*already visited, non-parent\* neighbor as a cycle (revisiting the parent itself is expected and fine in an undirected graph - the edge back to where you came from isn't a real cycle, just how undirected edges work). If the DFS completes without finding a cycle and reaches every node, it's a valid tree.

### Code

```

def valid_tree(n: int, edges: list[list[int]]) -> bool:
    if len(edges) != n - 1:
        return False

    adj: dict[int, list[int]] = {i: [] for i in range(n)}
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)

    visited = set()

    def dfs(node: int, parent: int) -> bool:
        visited.add(node)
        for neighbor in adj[node]:
            if neighbor == parent:
                continue
            if neighbor in visited:

```

```

        return False
    if not dfs(neighbor, node):
        return False
    return True

if not dfs(0, -1):
    return False

return len(visited) == n

if __name__ == "__main__":
    tests = [
        (5, [[0, 1], [0, 2], [0, 3], [1, 4]], True),
        (5, [[0, 1], [1, 2], [2, 3], [1, 3], [1, 4]], False),
        (4, [[0, 1], [2, 3]], False),
        (1, [], True),
    ]

    for i, (n, edges, expected) in enumerate(tests, 1):
        got = valid_tree(n, edges)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

## 10. Pacific Atlantic Water Flow

10\_pacific\_atlantic\_water\_flow.py

### **Explanation**

Problem: Given a grid of heights representing an island (Pacific ocean borders the top and left edges, Atlantic borders the bottom and right edges), water flows from a cell to a 4-directional neighbor only if the neighbor's height is  $\leq$  the current height. Return every cell from which water can reach *both* oceans.

Source : LeetCode 417 - Pacific Atlantic Water Flow

Example:

Given a 5x5 height grid, return the list of [row, col] cells that can reach both the Pacific and Atlantic borders.

Idea: checking "can water starting here reach both oceans" for every single cell independently would mean running a full flood-fill from every one of the grid's  $O(\text{rows} \times \text{cols})$  cells - too slow. Reverse the question instead: start from the ocean borders and ask "which cells could water flow *backward* into, from here" - since flow requires non-increasing height forward, flowing backward from an ocean means walking to neighbors with height  $\geq$  the current cell (the exact reverse condition). Running one multi-source DFS/BFS from all Pacific-adjacent cells marks every cell that can reach the Pacific; same from all Atlantic-adjacent cells marks Atlantic-reachable cells. The answer is simply the intersection of those two visited sets - two multi-source traversals total, not one traversal per cell.

### **Code**

```

def pacific_atlantic(heights: list[list[int]]) -> list[list[int]]:
    if not heights or not heights[0]:
        return []

    rows, cols = len(heights), len(heights[0])
    pacific: set[tuple[int, int]] = set()
    atlantic: set[tuple[int, int]] = set()

    def dfs(r: int, c: int, visited: set, prev_height: int) -> None:
        if (r, c) in visited:
            return
        if r < 0 or r >= rows or c < 0 or c >= cols:
            return
        if heights[r][c] < prev_height:
            return
        visited.add((r, c))
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            dfs(r + dr, c + dc, visited, heights[r][c])

    for c in range(cols):
        dfs(0, c, pacific, heights[0][c])
        dfs(rows - 1, c, atlantic, heights[rows - 1][c])
    for r in range(rows):
        dfs(r, 0, pacific, heights[r][0])
        dfs(r, cols - 1, atlantic, heights[r][cols - 1])

    return [(r, c) for r in range(rows) for c in range(cols) if (r, c) in pacific and (r, c) in atlantic]

if __name__ == "__main__":
    heights = [
        [1, 2, 2, 3, 5],
        [3, 2, 3, 4, 4],
        [2, 4, 5, 3, 1],
        [6, 7, 1, 4, 5],
        [5, 1, 1, 2, 4],
    ]
    expected = {(0, 4), (1, 3), (1, 4), (2, 2), (3, 0), (3, 1), (4, 0)}

    got = {(r, c) for r, c in pacific_atlantic(heights)}
    status = "PASS" if got == expected else "FAIL"
    print(f"Test 1: {status} (got {sorted(got)}, expected {sorted(expected)})")

```